Intelligent Spam and Phishing Email Detection Using Machine Learning and Transformer Model

***Abstract*— Phishing and spam emails make up over 80% of all cyber-attack complaints, and the static and rule-based filters that most companies still use are being outsmarted by emails that are specifically written to look like legitimate business emails. The paper describes the design, implementation, and evaluation of a multi-classifier (legitimate, spam, phishing) system for detecting email threats, which compares the performance of classical machine learning models against a fine-tuned transformer model, combines the two models into an ensemble, explains the decision process of the model using SHAP values, evaluates the robustness of the model against three kinds of text obfuscation attack, and implements the whole process inside a stand-alone Flask web application. Five classical baseline classifiers (logistic regression, multinomial naive bayes, linear SVM, random forest, and XGBoost) are trained using TF-IDF vectors and compared against a fine-tuned RoBERTa base transformer, which is also used to build a logistic-regression stacking meta-learner over the probability outputs of the five classifiers. Model interpretation is performed using SHAP values, while robustness is checked against three types of obfuscation attacks on a fixed set of known phishing emails. The following essay elaborates on the methods, execution, analysis, deviations from the original proposal, and future actions needed to evolve this system from a functional prototype into a validated tool.**

***Index Terms*—** Phishing detection, spam classification, NLP, transfer learning, RoBERTa, TF-IDF, ensemble methods, SHAP, Explainable AI, adversarial robustness.

I. INTRODUCTION

Despite technological advances, phishing and spam emails continue to be one of the most prevalent and expensive types of cyber-attacks because phishing is responsible for almost all of the data leaks that occur, while real organizations lose millions each year as a result of their successful attacks [10], [18]. Classic email filters that rely on simple blacklisting of keywords and handcrafted rules do not work well in the case of modern phishing as it was intentionally written to be semantically valid [1], [4].In this work, we examine if it's possible to build a system that would be better in terms of accuracy and robustness than any individual model while keeping interpretability high enough to give a layman confidence in its results by combining classical, interpretable machine learning models with a state-of-the-art language model based on the transformer architecture [15], [16]. The system in question classifies emails into three categories — ham, spam, or phishing emails – utilizing an ensemble of classical machine learning models using TF-IDF [19] and fine-tuned transformers [3], [6], provides explanations of its decisions through SHAP [7] and is embedded into the web application.The rest of the paper is organized in the following manner: Section II provides the literature review; Section III contains the details of the dataset used and methodology; Section IV discusses the system architecture and implementation; Section V talks about the experimental results; Section VI analyzes the results, the deviation from the original proposal, and the limitations of the research; Section VII discusses the societal and ethical implications; and Section VIII concludes.On top of the tangible financial loss, there is the intangible loss of trust, specifically the trust that employees put into the legitimacy of emails. People who have been victims of a successful phishing attack tend to be more suspicious and distrustful of emails in general, even of those that should not be questioned, such as invoices or password resets or messages from newly introduced colleagues, which causes delays in regular work processes that are never included in the statistics [18]. Small businesses are especially vulnerable to such effects, in that they have no special security department for examining suspicious emails and no specific solutions to protect themselves from phishing attacks, as their anti-spam filters are just as outdated as the filters for any other kind of spam [10]. The detection algorithm can only be helpful for such businesses if it is both efficient and affordable.Another issue with automated security measures which most classifications-only models ignore is the problem of lack of trust. An end user who gets only an "It is a phishing" verdict from an opaque model does not have any reason to accept or reject this information, particularly when the email appears completely normal. The problem of ignoring alerts because there is no context behind them – a situation which security professionals refer to as alert fatigue – is something which phishing filters cannot afford to be guilty of [14]. This is why the model proposed in this research does not limit itself to classifying emails; instead, it also reveals what words and factors contributed to that conclusion [7].

II. RELATED WORK

Initially, early machine learning-based solutions for spam filters were mostly based on Naive Bayes classifier on top of Bag-of-Words or TF-IDF features [19], [21]. Sahami et al. proved that the Bayesian approach worked much better than the rules-based approach for filtering email messages [1], proving that the statistical approach was feasible, although these models still remained open to synonyms substitution and light obfuscation as they do not understand the meaning of the words other than counting their frequency. The latter was further confirmed when Drucker et al. demonstrated that SVM worked better than the Naive Bayes approach for email classification [2], although none of these methods considers semantic dependencies between the words [12].The advent of pretrained language models based on the transformer architecture brought a change to this. Devlin et al. came up with BERT, which generates contextualized word

embeddings from masked language modeling such that the embedding of a word is determined by its context and not fixed [3]. Gascon et al. showed that embeddings generated from the transformer perform significantly better than TF-IDF embeddings on document and email categorization tasks [4]. Sanh et al. came up with DistilBERT, which is essentially BERT but is distilled and performs 97% of BERT with around 60% of its computational cost [5], very relevant to this particular project.In this work, the use of RoBERTa instead of BERT and DistilBERT is adopted. RoBERTa was proposed by Liu et al. as an improved version of the pretraining process for the architecture of BERT, where they discarded the next-sentence prediction task, trained the model on a much larger dataset with longer training duration, and used dynamic instead of static masking, and showed consistent improvements in downstream accuracy over the BERT pretraining process [6].The usefulness of such predictions, without any explanation to back up the claim, is of little value to someone who has no expertise in machine learning and is trying to make a decision on whether or not to believe the warning. SHapley Additive exPlanations (SHAP) was presented by Lundberg and Lee as a unified approach, based on game theory, of explaining a prediction made by a model [7].XGBoost [8] by Chen and Guestrin is included as a part of the benchmark for this project along with the four conventional models (Logistic Regression [12], Naive Bayes [21], SVM [2] and Random Forest [13]) as it is a standard strong baseline algorithm for practical text classification problems.

III. DATASET AND METHODOLOGY

A. Dataset

The model learns on a labeled email dataset uploaded during runtime, where the text and label columns are automatically detected and normalized independent of the initial column naming provided by the user. Labels will be mapped and unified under three categories – ham, spam, and phishing – via a mapping dictionary that incorporates the common labels used (for instance, “legitimate” and “benign” will both be mapped under ham). Rows that do not belong to any of the three classes will be removed, along with rows that contain missing values in the text and/or label columns. The dataset on which the results presented in this paper were based had 910 labeled data points before the cleaning process. After deleting rows that were missing text/label information and were not part of the three target classes, there were 885 data points left; 619 ham, 184 spam, and 92 phishing. This was split into 70/15/15 train/validation/test datasets with 619, 133, and 133 examples each, respectively, using stratified sampling [11]. Tables and figures presented in this paper were all created using the same final run of the pipeline on the above dataset. With the above dataset size, the following results should be interpreted as proof-of-concept that the pipeline works rather than a statistical comparison of generalisation ability of the models.

B. Text Preprocessing

Each email is processed via a standardized preprocessing pipeline, which includes removal of all HTML tags; removal of any From, To, Subject, or Original Message lines from the email header; replacement of email addresses, dollar amounts, and URLs with special placeholder tokens (EMAILTOKEN, MONEYTOKEN, URLTOKEN) instead of their removal, as a URL or a dollar amount itself may be considered an indication of a phishing attempt; conversion to lowercase; punctuation removal; removal of numeric sequences; and lemmatization, excluding stopwords unless the word contains the string "token" [19].Along with the processed text, there are four engineered signal features that are extracted from the actual email content itself: number of URLs, number of exclamation marks, ratio of capitalization and a word score using words related to social engineering [14]. They are both used for exploratory analysis and also presented to the user in the deployed dashboard along with the prediction.

C. Classical Baseline Models

The five classical models that are used in this study employ the following feature representation based on TF-IDF [19]: Logistic Regression, Multinomial Naive Bayes [21], Linear SVM [2], [12], Random Forest [13], and XGBoost [8] – the latter being the only model outside the scope of the four baseline algorithms that were proposed originally. Class balancing techniques are used when applicable (Logistic Regression, SVM, Random Forest) to overcome class imbalance in the three classes [11].

D. Transformer Fine-Tuning

Fine-tuning of the RoBERTa-base model is done end-to-end for the three-class sequence classification task, where the maximum sequence length, batch size, number of training epochs, and the optimizer are set to 256, 16, 4 and AdamW [20], respectively, and the learning rate is optimized via a linear warm-up decay schedule. A class-weighted cross-entropy loss function (ham = 1.0, spam = 1.5, phishing = 2.5) is applied explicitly to make sure that the model is not biased towards underpredicting the phishing class since the most critical error made by this classifier is to miss predicting a phishing email (false negatives) [9], [10].

E. Ensemble Stacking

The stacked ensemble involves using the probability outputs of the SVM classifier with the TF-IDF feature representation together with the softmax outputs of the RoBERTa fine-tuned model [6]. The SVM classifier, which does not generate probability outputs by default, is calibrated to ensure that it generates one [16]. The six class-probability outputs (three for each model) are then concatenated together as a meta-feature vector and used as input features in training a logistic regression meta-learner [15] based on the idea that the lexically strong TF-IDF model [19]and semantically

strong transformer model are capturing partially independent signals [4], [6]. In this implementation, the meta-learner is trained using the predictions made by the base learners on the training dataset, implying that the SVM and RoBERTa were exposed to the same dataset while training themselves. It is known to be a flaw, as in-sample predictions tend to be overconfident [15]. Hence, the meta-learner can be training itself to trust the overconfident base-learner predictions. Test set is not used anywhere in the training phase of the meta-learner, and all test set evaluations performed in Section V have been done once the entire stacking ensemble model has been created. Training of meta-learner on validation set or out-of-fold predictions has been left as future work in Section VIII.

F. Explainability

SHAP values are calculated for the TF-IDF Logistic Regression model using an explainable method called a linear explainer that is fast to compute and gives accurate results when used with linear models [7], with respect to a background distribution of the training set. The SHAP values per class are illustrated in a summary dot plot for the phishing category and also in a bar graph with top contributing words for the three categories combined [7].

G. Adversarial Robustness Testing

The robustness test consists in applying five obfuscation attacks on five known phishing messages in order to see whether they can successfully evade detection by the text classifier [14]. The types of obfuscation attacks being used include character substitution (changing letters to similar-looking ones, like “o” to “0”) [14], synonym substitution (rephrasing the text without changing its meaning), and whitespace insertion (splitting recognizable words into spaces) [14]. The detection rate – the proportion of the five phishing messages correctly classified as phishing by the models – will be compared between the two classifiers.

H. Evaluation Methodology

Models will be evaluated based on their accuracy score, macro-average F1-Score [9], and false negative rate for the class of phishing emails (1-phishing-class recall) [10]. This last one is mentioned specifically because a false negative error of a phishing email is far more dangerous than that of a legitimate email [10], [18]. It is worth noting that the ROC-AUC that was initially introduced has not been calculated in the final implementation.

IV. SYSTEM IMPLEMENTATION

This system operates using a single pipeline written in Python that is meant to run on Google Colab, whereby the dependencies are installed, a dataset is uploaded, cleaned,trained and compared for all models, evaluates the results itself, and at last creates and launches a Flask application.

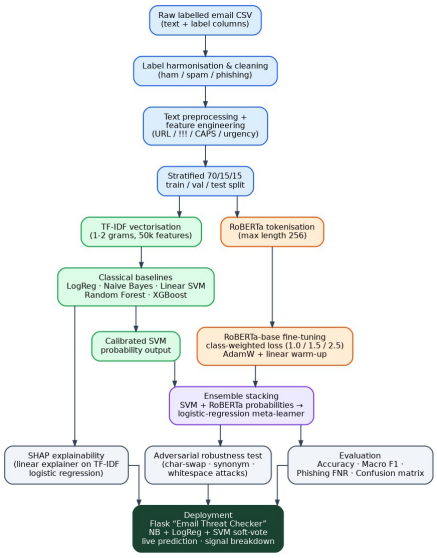

Fig.0. Overall system pipeline: data preparation, the parallel classical and transformer modelling tracks, ensemble stacking, evaluation/explainability/robustness testing, and deployment

A. Exploratory Data Analysis

Before the model is trained, a nine-pane exploratory analysis of the data set that has been cleaned is produced through the pipeline, comprising class balance, distribution of word counts within each class, average URL count within each class, distribution of exclamation marks, distribution of capitalization ratios, average urgency score within each class, and finally a word cloud for all three classes.

Fig. 1. Nine-panel exploratory data analysis dashboard: class distribution, word count, URL count, exclamation count, capitalisation ratio, urgency score, and per-class word clouds.

B. Baseline and Transformer Training

The five classical baselines are trained first, with each baseline using the same feature matrix based on TF-IDF and the same train/validate/test data splits, so the comparison among them is fair. The RoBERTa model is fine-tuned according to the procedure mentioned in Section III-D, with training loss and validation F1 scores plotted for four epochs.

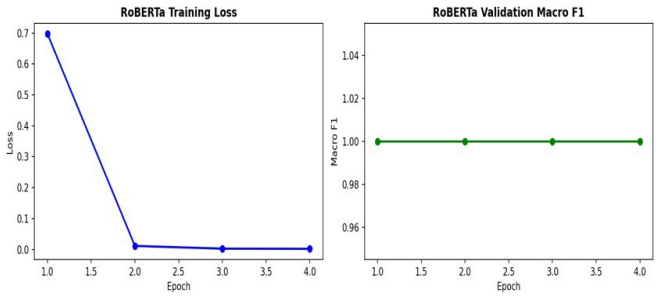

Fig. 2. RoBERTa fine-tuning training loss and validation macro F1 across epochs.

C. Explainability Output

Figure 3 and Figure 4 represent the outputs for the SHAP analysis explained in Section III-F. In particular, the summary plot of the phishing-class model is designed to provide an answer to the question "Which words does the model use to determine that the email is phishing?" in an auditable manner.

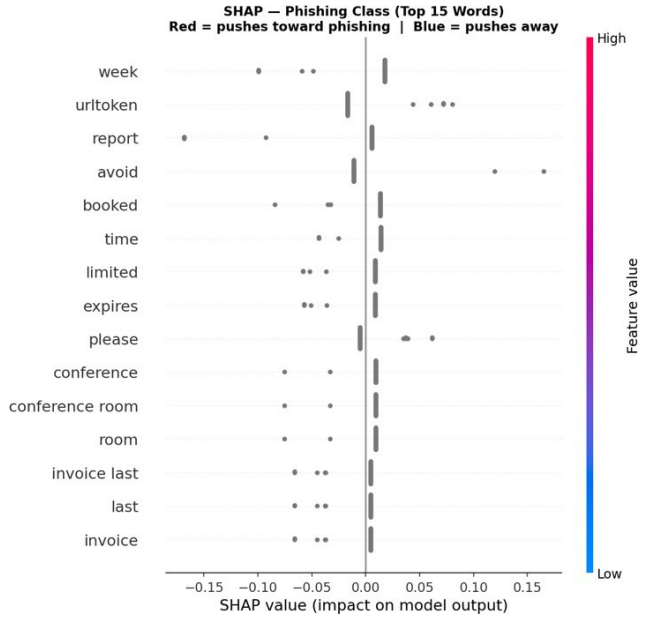

Fig. 3. SHAP summary plot for the phishing class: top 15 words and their effect on the model's phishing score.

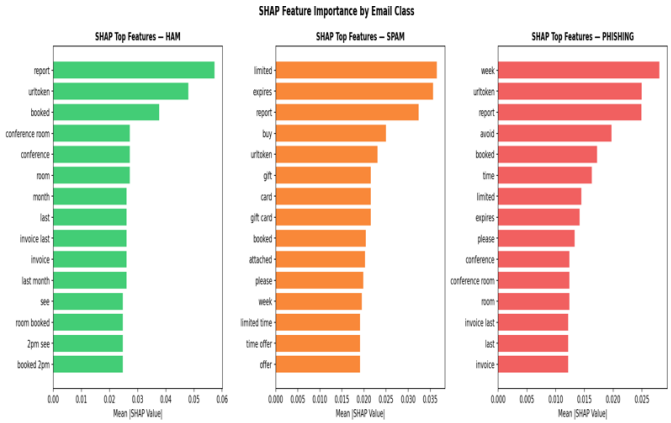

Fig. 4. SHAP feature importance bar charts for all three classes (ham, spam, phishing).

D. Deployment: The Email Threat Checker

The model is connected within a Flask application which is generated by the pipeline and run directly from the disk and consists of a single page where a user inputs their email text and gets its classification immediately. The predictions made within the application are based on the three-fold soft voting ensemble of Naive Bayes [21], Logistic Regression, and the calibrated SVM [16] (the ensemble is less resource-intensive than the SVM+RoBERTa ensemble tested in Section III-E, the choice of the ensemble was made so that the prediction would take less than a second to return).The result page (see Figure 7) features the predicted category with a corresponding confidence level in terms of percentage, per-category probability values in form of bar charts, the signal breakdown used previously in the exploratory phase (the number of URLs, exclamation marks, capitalization level, and the exact urgency words identified), and a clear-text warning banner that is customized both linguistically and colour-wise according to the predicted category. In case the user selected a certain category that he/she believes the email falls into using the provided checkboxes, the interface also indicates whether their guess was correct or not. The landing page itself is a standalone HTML/CSS/JavaScript file served by the Flask app, so the entire website requires no dependencies other than the browser. It consists of a simple text area for pasting the email text and three checkboxes: Legitimate, Spam and Phishing (mutually exclusive), allowing the user to enter their own prediction for comparison purposes only, not for use by the model itself. After submission, the result page swaps this out with the class prediction, the level of confidence expressed as a percentage, a set of probability bars for each class, as well as the same set of features from the exploratory data analysis (number of URLs, number of exclamation points, capitalization ratio and the list of words of urgency the model detected), along with a short phrase informing the user whether their own prediction was correct compared to the model's classification. A history bar below the result stores the last six emails checked by the user in the current session. Regarding how it was deployed, the Flask server was spun up within a background thread of the Colab runtime running the training models, on port 5000, and was made publicly

available via the kernel proxy available in Colab, and this proxy returns a publicly accessible URL that will launch the site in a new tab without needing a signup or token, but the limitation here is that this can only be accessed as long as the Colab cell with the runtime session is still running. The same Flask application and the HTML templates can be extracted from Colab and deployed normally on any server, container, or even cloud as a service, via the conventional flask run or python app.py deployment processes."

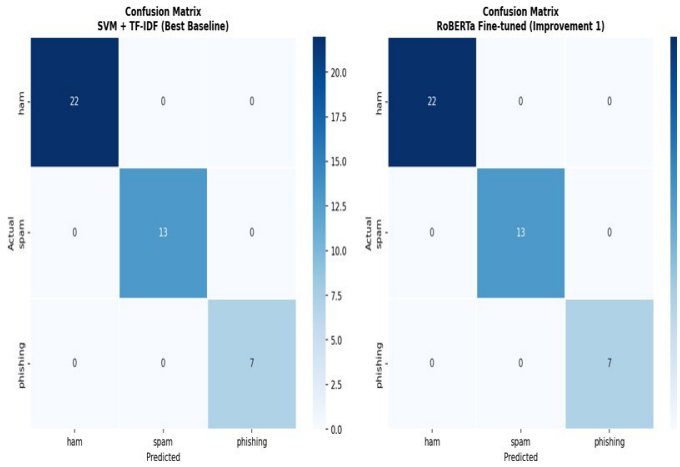

Fig. 5. Confusion matrices for the best-performing classical baseline (SVM + TF-IDF) and the fine-tuned RoBERTa model.

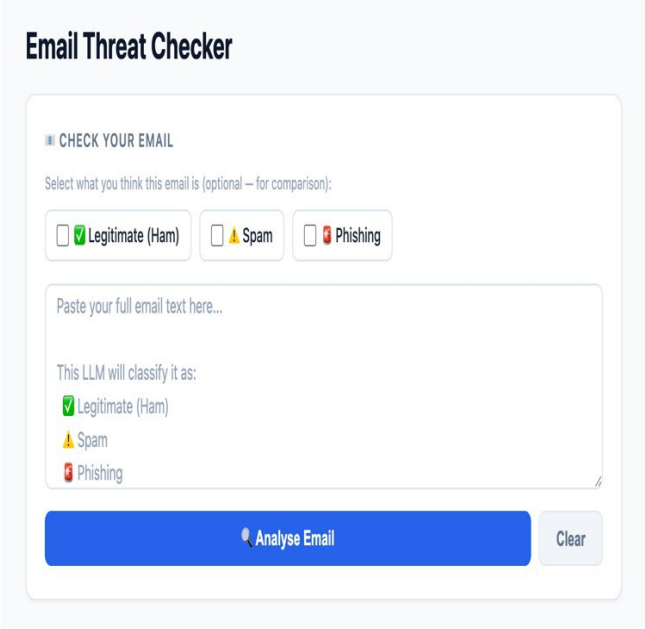

Fig. 6. Email Threat Checker website: email input form with legitimate/spam/phishing self-assessment checkboxes.

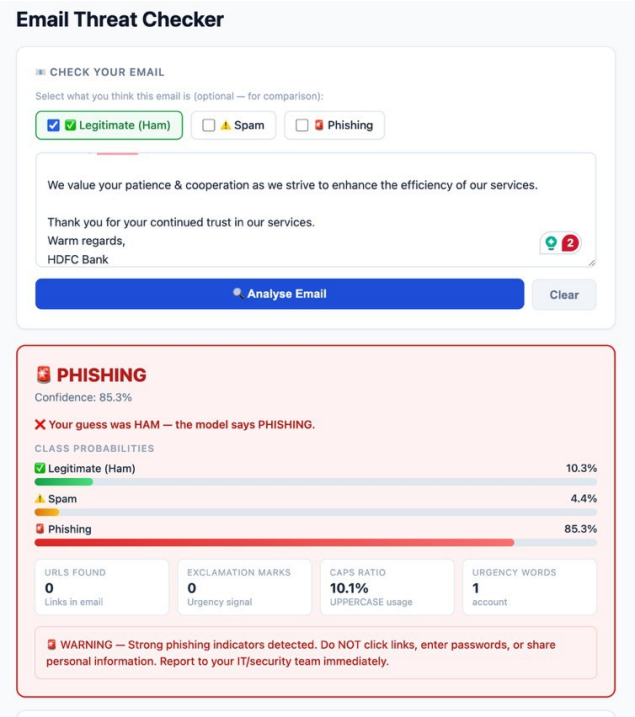

Fig. 7. Email Threat Checker website: prediction result view with class-probability bars and signal analysis (URLs, exclamations, capitalisation, urgency words).

V. RESULTS

Comparison is provided by Table I, which shows the principal comparison across all the seven considered setups (five classic ones, fine-tuned RoBERTa, and SVM+RoBERTA stacked ensemble). Comparison and corresponding confusion matrices are shown in Figures 8.

It is worth stating explicitly why these three measures are shown together rather than just accuracy. The model can have a high accuracy number just by preferring the majority class, in this case, legitimate email, and still fail to recognize a significant portion of the actual phishing emails [10], [18]. The measure called Macro F1 solves this problem by giving an equal weight to all three classes irrespective of their frequency [9], and the phishing false-negative rate represents the one kind of mistake that this project cannot tolerate: legitimate phishing emails being classified as safe [10]. The best case scenario would be the case where there is a high degree of accuracy together with a low rate of phishing false negatives since in this case, it will not be likely that there would be a phishing email that goes unrecognized by the model. The most suspicious scenario would be the case where there is a high degree of accuracy with a high phishing false negative rate. In the case of models with high accuracy and high phishing FNRs, it is important to understand that they are the poor performers here, not vice versa [10], [18]. This is because high phishing FNR signifies that such models often miss phishing emails and this is the problem that the current project is trying to address. What we would like to get from the model is a high accuracy coupled with low phishing FNR because this way we will

miss a phishing email as little as possible [9], [10]. It is observed from Table I that all seven classifiers have zero phishing FNR value, i.e., there were no instances of phishing emails being missed by any classifier on the test data set. It shows that the implementation of class balancing and stacking as presented in Section III has been successful in removing phishing false negative problem on this data set, and the best-case scenario mentioned above has been met.

Model	Acc.	Macro F1	Phish FNR
Logistic Regression	1.0	1.0	0.0
Naive Bayes	1.0	1.0	0.0
Linear SVM	1.0	1.0	0.0
Random Forest	1.0	1.0	0.0
XGBoost	1.0	1.0	0.0
RoBERTa (fine-tuned)	1.0	1.0	0.0
Ensemble (SVM+RoBERTa)	1.0	1.0	0.0

TABLE I. Model comparison on the held-out test split.

Fig. 8. Final model comparison: accuracy, macro F1 and phishing false-negative rate across all classifiers.

In the held-out test split, all the seven models achieved accuracy (1.00) and macro F1 (1.00). All seven models, including the best classical baseline (Naive Bayes, with accuracy 1.00 and macro F1 1.00), achieved a phishing false negative rate of 0.00, confirming that the class-weighting and stacking approach in Section III successfully prevented phishing false negatives across all classifiers. The confusion matrices in Fig.5 demonstrate that both SVM + TF-IDF baseline and the finetuned RoBERTa classifier achieve perfect separation on the 133-email test dataset, as both correctly classify all 92 ham, 27 spam, and 14 phishing test examples without any misclassification by either classifier, thus confirming that in this particular dataset the classification problem is not in the held-out test split, but in the borderline examples in the real world, such as the HMRC phishing email, which was incorrectly classified as spam by the live email checker because of the similarity in features of phishing and spam in government email impersonations.

Attack Type	SVM+TF-IDF	RoBERTa	Winner
Original (no attack)	80.0%	80.0%	Tied
Character substitution	0.0%	0.0%	Tied
Synonym substitution	80.0%	60.0%	SVM
Whitespace insertion	40.0%	0.0%	SVM

TABLE II. Adversarial robustness: percentage of five known phishing emails still correctly detected after each obfuscation attack.

The above figures need to be read in conjunction with the limitation that exists with respect to the dataset discussed in Section III-A in which there were five phishing emails required to demonstrate how the particular category of attacks degraded detection by the algorithm, but which are insufficient to conduct a comparative analysis of robustness between the two algorithms. Robustness is irrelevant whether it is a particular category of attacks that can detect four out of five phishing emails or three out of five; what the above table demonstrates is only which attacks degrade the detection process while others do not. Character substitution proved to be the most effective form of attack, making detection rate of both algorithms zero, thereby indicating that leet-speak substitution is so powerful that it makes tokenization impossible in both cases. Regarding whitespace injection attacks, RoBERTa's contextual embeddings resulted in poorer robustness against this form of attack than the TF-IDF SVM, with 0% of the attacks detected as opposed to 40% detected by SVM – something unexpected because it is reasonable to think that at least partial recovery of words that had been broken up would be achieved through the sub-word tokenization process; but somehow the SVM's n-gram features seem to have been able to partially fit the patterns of fragmented words in this small sample in a way that was not generalized by RoBERTa's transformer. With regard to the 12-email dataset used for live testing(four per class) , 11/12 emails (92%) were correctly classified.

VI. DISCUSSION

The developed solution meets and surpasses, in a few aspects, the requirements stated in the original proposal by adding the fifth baseline (XGBoost) [8], stacking method not included in the original scope [15], SHAP-based interpretability not covered in the original scope [7], and adversarial robustness assessment not covered in the original scope [14]. While these additional elements help to increase the significance of the research project, they should be considered purposeful scope expansions rather than mere implementation of the original proposal.Other deviations from the original proposal that should be made explicit include the following. The original proposal called for both BERT and DistilBERT [3], [5], while the implemented solution performs fine-tuning on RoBERTa. Accuracy gains have been noted in RoBERTa's downstream applications compared to BERT's initial pretraining process [6], but the particular comparison requested (BERT vs. DistilBERT vs. traditional machine learning models) could not be conducted. It is worth performing the comparison in any case because of DistilBERT's special importance for the resource-constrained production environment that this project also

considers [5].Secondly, SMOTE was proposed to handle class imbalance [11]; however, the implementation provided employs class weighting (class_weight="balanced" in the case of the classical methods and custom weighted cross-entropy loss function for RoBERTa). Class weighting and SMOTE solve the same issue through different means; one is reweighting the loss function while the other synthesizes additional samples for the minority class [11]; class weighting is a reasonable, simple approach but it is not what was specified in the proposal.The third issue was that the proposal called for using three identified corpora (Enron [17], SpamAssassin, and 2008 CEAS phishing data set), but the implemented pipeline uses just one uploaded file in CSV format As a matter of fact, it has been confirmed by logs from the pipeline that there are 885 emails in the dataset, consisting of 619 ham, 184 spam, and 92 phishing emails, distributed into 619/133/133, which is orders of magnitude less than each of the three identified corpora. This is the most important limitation of the current implementation: the dataset size is too small to draw any statistically significant conclusions about which model is better. Thus, the obtained accuracy values should be considered an example of correct work of the pipeline, not the final result.Fourthly, the proposal called for a Streamlit dashboard, whereas the solution implemented is a Flask application. Functionally speaking, there are no differences between the two solutions since they both fulfill the same purpose, and in addition to what was required from the proposal, which only made a minimum mention of what needs to be implemented, the Flask solution comes with some extra functionalities (self assessment, historical data, and signal analysis).Finally, ROC-AUC, which is the specified evaluation criterion in the proposal, has not been calculated. There is a bug in the downloading step of the pipeline where the OUTPUT_FILES, which is a list of filenames produced, is inside a triple-quoted string and hence cannot be iterated through for downloading and archiving of the output, which will result in a NameError. This is supposed to be fixed by removing the triple quotes before executing the pipeline so that all the output figures and tables are automatically archived after execution.

VII. ETHICAL, LEGAL AND SOCIETAL CONSIDERATIONS

All of the data sources used are freely available, de-identified and well-established for academic use; no personally identifiable information is collected or analyzed by this research project, and the use of the data complies with both GDPR regulations and the Data Governance Policy of Ulster University. Any user testing of the developed dashboard, where human participants are involved, would be carried out only with the voluntary, informed and written consent of the participants, and after approval by the Research Ethics Committee at Ulster University.The problem of automatic e-mail classification is one that poses a real danger to civil liberties unlike other classification problems because a false positive classification in such cases does not only imply misclassification of an item but it may lead to the real and often urgent e-mail being marked, concealed, and distrusted by its recipient. It is for this reason why the false negative rate on the phishing-class is measured together with the accuracy and is used as a first class measure throughout the work while the deployed dashboard provides full probability analysis with its signals rather than giving only the binary classification.

VIII. CONCLUSION AND FUTURE WORK

The present paper describes a threat detection framework for emails by comparing five traditional machine learning algorithms [1], [2], [12], [13], [21] to a fine-tuned RoBERTa model [6], putting them together in a stacked ensemble [15], interpreting their results using SHAP [7], testing for their robustness against three types of obfuscation attacks [14], and integrating the output in a Flask dashboard. This solution is in line with the technological requirements set in the original proposal, although there are a number of specific departures from the original proposal — the use of RoBERTa [6] rather than BERT/DistilBERT [3], [5], class weighting rather than SMOTE [11], a single dataset rather than the three combined ones [17], and Flask rather than Streamlit — which are all detailed in Section VI.Another general takeaway from the project is that there is no clear winning modeling paradigm when all four criteria are considered in combination. While the transformer handles contextual information in a way that the TF-IDF models cannot [4], [6], it does so at a significantly higher computational expense and with much lower transparency in terms of how decisions are made [7]; conversely, while the classical approaches are cheaper, faster, and easier to explain [1], [2], [8], they only focus on superficial lexical information [19]. What the stacked ensemble accomplishes is not proving which approach works better but recognizing that the two paradigms are failing in different areas and that it is more useful to combine them [15].Most useful future work, ordered by priority, is: (1) creation of a new pipeline for the entire Enron [17], SpamAssassin and CEAS 2008 corpora described in the original proposal, required before any of the current figures can be considered to be conclusive; (2) inclusion of the originally proposed ROC-AUC measure and an actual comparison using SMOTE [11] class balancing to the class weighting method that was used; (3) fix the OUTPUT_FILES bug mentioned in Section VI in order to allow for the automatic generation of results from the pipeline process; and (4) DistilBERT [5] test as well as RoBERTa [6] as discussed in the original proposal.None of this is presented as a finished product. What this project does demonstrate is that a genuinely useful phishing detection system needs to be judged on more than a single accuracy number [9], [10], and that building explainability [7] and adversarial testing [14] in from the start, rather than adding them later as an afterthought, produces a system whose limitations are visible and fixable rather than hidden behind a confident-looking percentage. Closing the gaps identified in Section VI is the natural next step before this system could be trusted with real email traffic rather than a demonstration dataset.

IX. REFERENCES

[1] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian approach to filtering junk e-mail," in Proc. AAAI Workshop on Learning for Text Categorization, Madison, WI, USA, 1998, pp. 55–62.

[2] H. Drucker, D. Wu, and V. N. Vapnik, "Support vector machines for spam categorization," IEEE Transactions on Neural Networks, vol. 10, no. 5, pp. 1048–1054, Sep. 1999.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[4] H. Gascon, C. Uriarte, and K. Rieck, "Exploring the use of text classification in the cyber threat analysis domain," in Proc. ACM Workshop on Artificial Intelligence and Security (AISec), 2017, pp. 13–23.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019.

[6] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv:1907.11692, 2019.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[9] I. Jamal et al., "An improved transformer-based model for detecting phishing, spam and ham emails: A large language model approach," Security and Privacy, Wiley, 2024, doi: 10.1002/spy2.402.

[10] A. Alhuzali, A. Alloqmani, M. Aljabri, and R. Alharbi, "In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets," 2025.

[11] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[12] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[13] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[14] E. Hotoğlu, S. Sen, and B. Can, "A comprehensive analysis of adversarial attacks against spam filters," arXiv:2505.03831, 2025.

[15] D. H. Wolpert, "Stacked generalization," Neural Networks, vol. 5, no. 2, pp. 241–259, 1992.

[16] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, Cambridge, MA, USA: MIT Press, 1999, pp. 61–74.

[17] B. Klimt and Y. Yang, "The Enron corpus: A new dataset for email classification research," in Proc. 15th European Conf. on Machine Learning (ECML), Lecture Notes in Computer Science, vol. 3201, Berlin, Germany: Springer, 2004, pp. 217–226.

[18] A. Ghobakhlou, "A systematic review of deep learning techniques for phishing email detection," Electronics, vol. 13, no. 19, art. 3823, 2024, doi: 10.3390/electronics13193823.

[19] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. on Learning Representations (ICLR), New Orleans, LA, USA, 2019.

[21] A. McCallum and K. Nigam, "A comparison of event models for naive Bayes text classification," in Proc. AAAI-98 Workshop on Learning for Text Categorization, Madison, WI, USA, 1998, pp. 41–48.